RunBaza Л5 Руководство к защите

Hamuev Oleg | CR-232 | Leah A.

Кратко что сделали

RunBaza теперь работает с собственным сервером на TypeScript и Hono. Вход и регистрация настоящие: сервер создаёт аккаунт и проверяет пароль. Забеги, участие, поездки и профиль читаются и сохраняются через HTTP. SQLite хранит записи после закрытия приложения и перезапуска сервера.

Что требовалось по лабораторной

Поднять backend с JWT и SQLite, добавить сущности своей темы, связать формы auth с API и хранением token. Реализовать HTTP-репозитории вместо mock, сохранив интерфейсы Л4. Обработать сетевые ошибки, неверный ответ сервера и 401. Добавить серверный поиск и страницы. Показать всё на iOS и доказать замену источника через Git.

Что изменилось относительно Л4

В Л4 записи жили в памяти mock. Теперь источником служит API, а записи лежат в базе на Mac. Переключатель искусственной ошибки удалён: для показа сбоя действительно останавливаем сервер. Каталог догружает записи при прокрутке и отправляет поиск серверу. Знакомые формы и дизайн сохранены.

Вступление которое можно сказать

«В пятой работе я подключил RunBaza к своему backend на TypeScript и Hono. Экраны продолжают работать через модели и интерфейсы репозиториев из четвёртой работы. Изменился источник: вместо памяти используется HTTP и SQLite. Покажу регистрацию, сохранение участия, поиск, сбой сервера и автоматический выход при недействительном token».

Что важно не перепутать

Это локальный сервер для лабораторной. Каталог содержит демонстрационные события. Публичное бронирование, offline-кэш и production deploy пока не сделаны. Собственные 58 API-проверок прошли; официальный набор курса 37/37 не запускался, потому что его файлов нет. Не называй собственный набор официальным.

Стр. 2: запуск. Стр. 3: основной показ. Стр. 4: ошибки и сессия. Стр. 5: код. Стр. 6: устройство backend. Стр. 7: вопросы. Стр. 8: Git и помощь при проблемах.

1 Запуск приложения и сервера

Быстро на твоём Mac

Открой Finder, затем Рабочий стол / PAM / Lab5. Двойной клик по start_server.command запускает backend. Оставь это окно Terminal открытым. Вторым открой run_ios.command: он запускает Simulator и Flutter. Если появится список устройств, выбери iPhone, а не macOS или Chrome.

Через Terminal вручную

```
cd ~/Desktop/PAM/Lab2/server
pnpm install --frozen-lockfile
pnpm start
```

Нужны Node.js 24 или новее и pnpm 11.25.0. Если Terminal не знает pnpm, используй start_server.command: на этом Mac он находит доступный runtime. В новом окне Terminal запускается приложение:

```
cd ~/Desktop/PAM/Lab2/run_baza
flutter pub get --enforce-lockfile
open -a Simulator
flutter devices
flutter run -d <iphone-simulator-id>
```

Вместо <iphone-simulator-id> подставь ID симулятора из списка. API для iOS Simulator по умолчанию: http://127.0.0.1:8080. Это сервер на Mac. На физическом iPhone localhost указывал бы на сам телефон, поэтому нужны LAN-адрес и соответствующая настройка iOS.

Если запускаешь через Xcode

```
open ~/Desktop/PAM/Lab2/run_baza/ios/Runner.xcworkspace
```

Сначала сделай обычный flutter run без тестового API_BASE_URL. Затем в Xcode выбери Runner и iPhone Simulator, нажми Run. Backend должен продолжать работать в отдельном Terminal. Встроенной обычной shell-консоли для этих команд в Xcode нет: используй приложение Terminal.

Первый вход

Готового логина нет. Нажми «Создать профиль», введи новое имя и email, пароль минимум 8 символов и его подтверждение. Запомни введённые данные. Повторная регистрация того же email даст ошибку; для существующего аккаунта используй «Войти».

2 Что показывать в приложении

Шаг 1 Регистрация и каталог

Создай аккаунт. После ответа сервера откроется главная. Скажи: «Теперь регистрация действительно записывает пользователя в SQLite и сохраняет token. Если сервер выключен, успешного входа не будет».

Шаг 2 Страницы

Открой «Забеги». Первая порция содержит 20 событий. Прокрути вниз: подгрузятся ещё 8. Скажи: «Телефон получает список частями. В запросе есть page и limit. Повторная загрузка защищена от дублирования».

Шаг 3 Поиск

Введи Iaşi в поиск. После небольшой паузы увидишь подходящие события. Очисти строку, попробуй фильтр страны. Скажи: «Это серверный поиск. В SQL фильтруются записи, а Flutter получает только результат текущей страницы».

Шаг 4 Создать участие

Открой Chişinău Marathon. Нажми закладку для личного плана, выбери 10 км, темп 5:40 и видимость. Подтверди. Перейди в «Мой план», затем в профиль. Скажи: «Форма вызвала PlanModel, репозиторий отправил POST /plans, сервер присвоил ID и сохранил запись. Модель обновила связанные экраны».

Шаг 5 Изменить и удалить

Открой то же участие, поменяй дистанцию на доступную в этом забеге, сохрани. Число участий не увеличится: это PUT с прежним ID. Затем удали запись через закладку с подтверждением. Скажи: «DELETE удаляет личное участие. Забег остаётся в общем каталоге».

Шаг 6 Профиль и поездки

Измени город или темп в профиле, сохрани. По желанию создай предложение водителя или запрос пассажира. Это тоже запись через HTTP. Здесь пока личные записи аккаунта, а не реальное бронирование мест между пользователями.

Что осталось знакомым

Навигация Л3, формы и проверки ввода, Provider/ChangeNotifier из Л4 и оформление экранов сохранены. Изменения интерфейса нужны только для auth, удаления диагностики, поиска и страниц.

3 Сбой сервера и управление token

Показать ошибку сети

В Terminal сервера нажми Ctrl+C. В приложении открой «Забеги» и нажми обновление. Появится сообщение об ошибке и «Повторить». Скажи: «Это реальная недоступность сервера. Offline-кэша в Л5 нет, поэтому старые данные не выдаются за новый ответ».

Показать retry

Снова запусти start_server.command или pnpm start в той же папке. В приложении нажми «Повторить». Список вернётся. База не удалялась: аккаунт и участие сохраняются.

Показать неверный пароль

В профиле выбери выход и подтверди. На входе укажи свой email, но неправильный пароль. Дождись завершения индикатора: появится русское сообщение. Скажи: «Сервер прислал UNAUTHORIZED. Текст для пользователя выбирает apiMessage, английское message не показывается».

Показать восстановление сессии

Войди правильно. Полностью закрой приложение через переключатель приложений Simulator и открой его снова. Оно откроет защищённую часть без нового ввода. Скажи: «Token пережил перезапуск в shared_preferences. Данные снова загружаются по сети, пароль локально не сохранялся».

Показать именно 401 при истёкшей сессии

Оставаясь авторизованным, останови сервер. В той же папке запусти его с другим секретом:

```
JWT_SECRET=defense-alternate-secret pnpm start
```

Нажми обновление каталога. Старый JWT не пройдёт проверку подписи, сервер вернёт 401. Приложение удалит token и откроет вход. Войди снова: аккаунт находится в той же базе, поэтому пароль продолжает работать. Token будет уже подписан новым секретом.

Для следующей защиты используй один и тот же секрет. Возврат к секрету по умолчанию снова сделает token от alternate-secret недействительным. Это ожидаемое поведение, а не потеря пользователя.

4 Где открыть код и как объяснить

Открыть папку и конкретные файлы

```
cd ~/Desktop/PAM/Lab2/run_baza
open -a Xcode lib/data/http_repository.dart
open -a Xcode lib/data/api_client.dart
open -a Xcode lib/data/token_store.dart
open -a Xcode lib/main.dart
```

Вставляй команды в Terminal. Один путь к .dart без open не открывает файл: shell пытается его выполнить. Для удобной работы можно открыть всю папку run_baza в VS Code. UI лежит в lib/screens, состояние в lib/models, сеть и сущности в lib/data.

Начни с HttpRepository.create

```
Future<Result<T>> create(T draft) => api.request(
  'POST',
  path,
  successStatus: 201,
  body: toJson(draft),
  decode: (json) => fromJson(json as Map<String, dynamic>),
);
```

draft: поля из формы. toJson: превращает их в JSON без ID. POST: просит сервер создать запись. 201: ожидаемый успешный статус. fromJson: превращает ответ, уже с серверным ID, обратно в объект Dart. Result возвращает либо Ok с записью, либо Err с понятным сообщением.

Потом открой ApiClient.request

Он добавляет Authorization: Bearer, отправляет запрос с timeout 8 секунд, проверяет HTTP-статус и только потом разбирает JSON. 204 пропускает jsonDecode: тело удаления пустое. SocketException, ClientException и TimeoutException дают сетевую ошибку, FormatException и TypeError дают ошибку формата.

Дальше TokenStore и main

TokenStore.save вызывается после успешного auth. read используется при запуске и перед защищённым запросом. clear вызывается при logout и 401. В main выбирается Repositories.http; модели получают интерфейсы через Provider. Они не знают URL сервера.

5 Как устроен backend

```
cd ~/Desktop/PAM/Lab2/server
open -a Xcode src/app.ts
open -a Xcode src/database.ts
open -a Xcode src/schema.ts
```

index.ts

Точка запуска. Читает PORT, HOST, DB_PATH и JWT_SECRET, открывает SQLite и запускает Hono. По умолчанию порт 8080 и файл data/app.db. Папка данных не публикуется в Git.

app.ts

Здесь маршруты /auth/register и /auth/login, затем защита JWT, CRUD и файлы. Перед записью сервер проверяет JSON. В запросах к личным таблицам используется user_id из проверенного token, а не ID пользователя из присланного тела.

database.ts

Создаёт таблицы users, notes, races, plans, offers, requests, profiles, files. user_id связывает запись с аккаунтом. race_id связывает участие или поездку с забегом. SQLite поддерживает внешние ключи. Seed добавляется в пустой каталог, а не сбрасывает базу при каждом старте.

schema.ts

Zod проверяет типы и ограничения полей. Дополнительная проверка в app.ts сверяет дистанцию участия с самим забегом, запрещает дубликат участия и смену account email через редактирование профиля.

Пароли и token

Пароль хранится на сервере как scrypt-хеш с солью. JWT подписывается HS256, содержит ID пользователя и срок 24 часа. При logout удаляется локальный token; серверного списка отозванных token здесь нет.

Файлы

POST /files принимает multipart с полем file, максимум 5 MiB. GET /files/:id доступен владельцу. Attachment и CSP ограничивают выполнение загруженного HTML в браузере. В интерфейсе RunBaza нет file picker: изображения маршрутов рисуются, а upload проверен на API.

6 Вопросы преподавателя и ответы

Что такое три слоя отказа

Транспорт: сервер недоступен или истёк timeout, ловим конкретные исключения. HTTP: ответ пришёл с 400/401/404/409/500, читаем statusCode. Данные: успешный ответ содержит неверный JSON или тип, ловим FormatException/TypeError. Во всех случаях UI получает Err.

Какое исключение бросает http при 404

Сам по себе 404 не обязан бросать исключение. Мы проверяем statusCode, переводим код ошибки через apiMessage и возвращаем Err. Это второй слой.

Почему нельзя разбирать JSON после DELETE

Успешный DELETE вернул 204 и пустое тело. jsonDecode пустой строки выбросит FormatException. Поэтому возвращается Ok<void> без декодирования.

Почему чужая запись отвечает 404

Так сервер не раскрывает, существует ли чужой объект. Ответ 403 мог бы подтвердить его наличие. Проверка владельца входит в SQL-запрос.

Почему старые fetchAll всё ещё компилируются

Новые параметры необязательные и именованные, с page=1 и limit=20. Поэтому прежний вызов без аргументов остаётся корректным.

Что происходит при двух быстрых поисках

У модели есть номер запроса. Поздний ответ предыдущего запроса игнорируется. loadMore имеет guard от параллельной догрузки, а новая страница не дублирует записи по ID.

Что хранится локально и безопасно ли это

По заданию хранится token в shared_preferences, пароль не сохраняется. Это учебная конфигурация. Для production потребуются защищённое хранилище, HTTPS, отдельный секрет, ограничения запросов и управление сессиями.

Это официальный сервер и набор conformance

Нет. По выбранному стеку сделан собственный Hono backend. Наши 58 HTTP-проверок и проверка перезапуска базы прошли. Официальный набор kit отсутствовал, его запуск не заявлен.

7 Git и помощь перед защитой

Показать что экраны не переписаны

```
cd ~/Desktop/PAM
git diff --stat 4c687e9 fa96d09 -- \
  Lab2/run_baza/lib/screens Lab2/run_baza/lib/models
git log --oneline c2b88af^..HEAD
```

4c687e9: версия Л4. fa96d09: HTTP уже подключён, но пагинация ещё не добавлена. В diff только login, register и profile. Формы auth допустимо менять, профиль теряет diagnostic switch. После этого e39b77f меняет каталог и CollectionModel ради поиска и страниц. Это отдельная задача, а не переписывание UI ради HTTP.

Команды проверки

```
cd ~/Desktop/PAM/Lab2/server
pnpm check
pnpm test
cd ../run_baza
flutter analyze
flutter test
```

Сервер не отвечает

Проверь, что окно pnpm start открыто и в нём нет ошибки занятого порта. В браузере открой http://127.0.0.1:8080/health: ожидается status ok. Затем обнови каталог.

Приложение ходит на порт 8090

Это порт интеграционного теста. Запусти обычный flutter run без API_BASE_URL или с --dart-define=API_BASE_URL=http://127.0.0.1:8080. Затем можно снова запускать тот же workspace из Xcode.

Email уже существует

Аккаунт остался в SQLite от прошлого показа. Нажми «Войти» с прежним паролем или зарегистрируй другой email. Не удаляй базу только ради этого.

После перезапуска вышло на login

Если менялся JWT_SECRET, это правильно. При обычном перезапуске с тем же секретом и той же базой token остаётся действительным до истечения 24 часов.

Последняя фраза

«Главный результат: UI и модели не зависят от того, где лежат данные. Я заменил реализацию репозитория, добавил настоящее управление сессией и проверил сбои. Следующая работа может добавить локальный кэш, не смешивая его с серверной базой».